

SightSync

What we built in Phases 1 to 3, the tools we used, and how the work flows

Written for: readers who are new to the project and want a plain-language overview rather than code details.

SightSync is an Android app meant to help visually impaired people understand their surroundings through the phone camera. The app looks at what the camera sees, works out what objects are there and how far away they are, and (in later phases) will describe this to the user with sound. This document explains the first three building blocks: the camera, the object recognition, and the distance sensing.

The project follows a nine-phase roadmap. Phases 1 to 3 are complete and have been tested on a real phone (a Nothing Phone 4a) as well as an Android emulator.

1. The big picture

Think of the app as a pipeline. Each phase adds one more step between 'the camera sees something' and 'the user hears about it':

Phase	What it adds	Plain-language meaning	Status
1	Camera pipeline	The app can open the back camera and receive a steady stream of images.	Done and tested
2	Object detection	The app recognises objects (people, chairs, bottles, etc.) in each image.	Done; model later switched to a more reliable version
3	Distance (depth)	The app measures how far away things are, using Google's ARCore depth sensing, with a fallback for phones that cannot do it.	Done and tested on a real phone

2. Tech stack (the tools we used)

A 'tech stack' is simply the list of tools, languages and libraries a project is built with. Here is what SightSync uses and what each part is for.

Area	Tool / technology	What it is used for
Language (app)	Kotlin	The language the Android app itself is written in.
Platform	Android (minimum Android 7.0, target Android 15)	The phone operating system the app runs on.
Build system	Gradle with the Android Gradle Plugin	Turns the source code into an installable app file (APK). Run with <i>gradlew assembleDebug</i> .
IDE	Android Studio / VS Code	Where code is written and edited.
Camera	CameraX 1.3.4 (Google)	Simple, reliable access to the phone camera for preview and frame analysis (Phases 1 and 2).
Depth sensing	ARCore 1.56.0 with Depth API (Google)	Google's augmented-reality library; the Depth API gives a distance value for each part of the image (Phase 3).
On-device AI	TensorFlow Lite 2.16.1 (+ GPU support libraries)	Runs the object recognition model directly on the phone, without needing internet.
Object model	YOLOv8n (Ultralytics), trained on the COCO dataset	A small, fast neural network that recognises 80 everyday object types.
Graphics	OpenGL ES 2.0 via Android GLSurfaceView	Draws the live camera image on screen when ARCore is in control of the camera (Phase 3).
Model conversion	Python 3.11, Ultralytics, ONNX, onnx2tf, Docker (Linux container)	Converts the trained model from its original format into the phone-friendly TensorFlow Lite format. Runs once, on the development computer.
Testing devices	Android Emulator (API 35) and Nothing Phone 4a (Android 16)	The emulator is for quick checks; the real phone is for anything that depends on real sensors or real scenes.
Version control	Git and GitHub (private repository)	Keeps a history of every change and a backup of the project. Nothing is committed without an explicit request.
Device debugging	Android Debug Bridge (adb), logcat	Installs the app on the phone and reads its log messages, which is how most problems were found.

3. What each phase built

Phase 1 - Camera pipeline

Goal: get a steady stream of camera images into the app, safely and reliably.

- Asks the user for camera permission the first time the app opens.
- Shows a live camera preview on screen.
- Receives a fresh image about every 33 milliseconds (around 30 per second) and counts them, reporting the frame rate in the log.
- Drops old frames rather than building up a backlog, so the app never falls behind the camera.

Key files: MainActivity.kt (the main screen), activity_main.xml (the screen layout), AndroidManifest.xml (permissions and app settings).

Phase 2 - Object detection

Goal: tell the user what objects are in front of the camera.

- A pre-trained YOLOv8n model is converted once, offline, into a TensorFlow Lite file and bundled inside the app.
- For each camera image, the app resizes it to the model's input size, asks the model for its guesses, and keeps only guesses that are confident enough (above 40 percent).
- Overlapping boxes for the same object are merged (a step called non-maximum suppression), so each object is reported once.
- A transparent drawing layer (DetectionOverlayView) can draw boxes and labels on top of the camera preview.

What we learned: converting the model was the hardest part of the whole project. The standard conversion tool failed on Windows in several ways and had a dependency that had been removed upstream. The fix was to run the conversion inside a Linux container (Docker), supply a replacement for the removed file, and verify every output step by step. In Phase 2, testing was limited to an emulator whose camera shows no real scene, so the model's real accuracy was not yet confirmed.

Important correction found in Phase 3: the smaller 'int8' version of the model, which is the one that would run fastest, turned out on the real phone to produce no usable results. The cause was in how the model was compressed. The fix in use today is the full-precision (float32) version of the same model, which works correctly at about 2 to 3 frames per second on the phone. Making the faster version work properly is a deferred improvement.

Phase 3 - Distance (depth) with ARCore

Goal: know roughly how far away things are, in millimetres, for every part of the view.

- The app first tries to start ARCore. If the phone supports ARCore's Depth feature, ARCore takes over the camera and provides both the camera image and a depth map.
- The depth map is a grid of distance readings. The app summarises it (centre, average, nearest and farthest distance) and logs it.
- If the phone cannot do depth, or ARCore is missing, the app automatically falls back to the Phase 2 camera-and-detection pipeline. Nothing crashes; the user just gets detection without distance.
- The camera image is drawn on screen by the app itself, so the user always sees a live picture.

Bugs found and fixed during testing:

- The ARCore install prompt could be triggered twice, skipping the user's consent screen. Fixed so the check runs at one well-defined point.
- The centre distance reading was always zero because the sampling step skipped the exact centre pixel. Fixed by reading the centre directly.
- The detection problem described in Phase 2 was traced and worked around (see above).

4. How the work flows

Every phase followed the same loop. This is the simple version of how a change goes from an idea to a tested app.

1. Plan

Decide what the phase must achieve and what 'done' looks like. The plan is written down in a plan file so it can be picked up again later.

2. Write the code

Changes are made to the Kotlin source files, or to the Python scripts for the model conversion.

3. Build

Gradle compiles the code into an app file. Compile errors are fixed before anything is installed.

4. Install

The app is pushed to the emulator or to the phone over USB using adb.

5. Run and watch the logs

The app is launched and its log messages are read. The log shows frame rates, detections, depth readings and any errors.

6. Fix and repeat

When something looks wrong, a temporary diagnostic is added, the app is rebuilt, and the log is checked again. Temporary diagnostics are removed afterwards.

7. Record

The plan file and the README in tools/ml are updated with what was built, what was learned, and what is still open.

8. Save to GitHub

Only when explicitly asked, the changes are committed and pushed to the private repository.

Model conversion workflow (done once, on the computer)

- Export the trained YOLOv8n model to the ONNX format (a standard exchange format) using Python.
- Convert ONNX to TensorFlow Lite inside a Linux Docker container, using onnx2tf.
- Check the resulting file (its input and output shapes and data types) with a verification script.
- Copy the verified model and its label list into the app's assets folder.

Testing workflow

- Emulator first, for quick install-and-launch checks and for the fallback path.
- Real phone for anything that needs real light, a real scene, or real movement (object recognition quality and depth readings).
- Each phase is only called done after the log shows the expected behaviour on the device.

5. Current status and what is next

Item	Status
Camera pipeline (Phase 1)	Working on the phone and emulator.
Object recognition (Phase 2)	Working on the phone with the float32 model; roughly 400 ms per frame (about 2 to 3 frames per second).
Distance sensing (Phase 3)	Working on the phone with ARCore Depth; fallback verified on the emulator.
Faster model (int8 with separate outputs)	Open improvement; not needed for the current phases.
Phase 4 onward	Not started. Next is sensor fusion and spatial logic (combining the detections and distances into a useful picture of the scene).

Note: the app is a development build. It is not yet a finished product and has no audio output yet; audio arrives in a later phase of the roadmap.